

CMPT-829 Project

Md Zahidul Islam, Student ID: 11370086

eqm971@usask.ca

Title: OpenGL Skeletal Animation using 3D Pose Estimation from RGB video.

Methods

In this project, I aim to use an off-the-shelf deep-learning based 3D human pose detector to get human skeletal joint co-ordinates and using that information for animating a graphics model in OpenGL. So, the overall methodology of the project can be divided into the following three parts- Pose Detection, Joint Angle Calculation, and Model Animation.

Pose Detection

For detecting pose from RGB video, I used MMPose[2] which is an open-source pose-detection toolbox based on PyTorch deep-learning library. The MMPose github repository hosts various deep-learning models for estimating 2D and 3D human joint coordinates. I used MotionBERT[1] which encodes pose information using a Dual-stream Spatio-temporal Transformer (DSTformer) deep neural network using the MMPose API for detecting 3D coordinates for each major joint in the human body. Given an RGB video, MMPose gives the 3D joint co-ordinates for the following 17 joints for each frame of the video.

0:'Root', 1:'RightHip', 2:'RightKnee', 3:'RightAnkle', 4:'LeftHip', 5:'LeftKnee', 6:'LeftAnkle', 7:'Torso', 8:'Neck', 9:'Nose', 10:'Head', 11:'LeftShoulder', 12:'LeftElbow', 13:'LeftWrist', 14:'RightShoulder', 15:'RightElbow', 16:'RightWrist'

The 'Root' joint refers to the mid-point of the 'RightHip' and the 'LeftHip'.

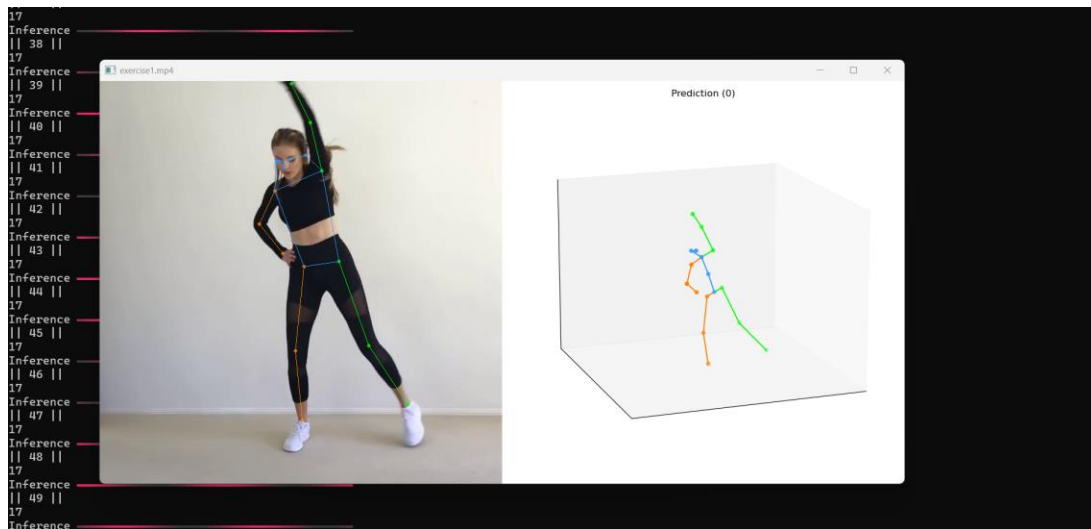


Figure 1: Estimating 3D joint co-ordinates from input RGB video collected from youtube.

For example, if the input video has 180 frames, then, the output pose information will be provided in an NumPy array with size 180x17x3. Figure 1 illustrates detecting 3D joint coordinates using MMPose pipeline.

Joint Angle Calculation

The joint co-ordinates derived from the pose estimation pipeline cannot be directly used to animate a humanoid model. It is because, usually graphical models are animated using a collection of “bones” which controls the movement of each vertex in the model. These bones are not actually rendered with the model, rather they just dictate how the vertices of the model should move. In biological entities, muscles move the bones. However, in graphics, the opposite seems to be true. Each of these bones have a position and an orientation, both of which can be represented using a 3-dimensional vector. The bones are arranged in a parent-children hierarchy and each bone’s position and orientation information are defined with respect to its parent. For example, the “LeftElbow” is a child of “LeftShoulder”. The orientation information basically defines what the body-pose will look like. That’s why we have to calculate joint angles for each joint from the output 3D pose co-ordinates from the pose estimation pipeline.

The methodology for calculating the joint angles or the orientation is as follows. First, we have to define the orientation of root joint using a vector. All the other children’s bones’ orientation vector will be defined with respect to the root. We use a recursive algorithm that uses the vector of a parent bone to calculate the vector of its children bone. As we know the joint co-ordinate of each bone, we can find an orientation vector a that points from a parent joint to its children joint. Then, using some mathematical calculations, we can find a rotation matrix that transforms the vector of the parent bone into the vector of the children bone.

The joint-angles or orientations are evaluated as Euler angles with the order of rotation axes as follows, Y->X->Z. This means that the rotation with respect to Y axis should be done first while manipulating the bones. However, we actually convert the angles into quaternions before passing them into OpenGL. The reason for this conversion is as follows. In OpenGL, while animating the bones we will need to interpolate for the frames in between the defined time-steps or key-points from the pose detection pipelines the framerate of OpenGL can be much higher than the framerate of the video. Euler angles are notorious for producing erroneous results during interpolation due to a phenomenon known as “Gimbal Lock”. That’s why, using a function from SciPy library we convert the Euler angles into quaternions before passing them into the 3D models. I stored the pose information (quaternions) in text file which the OpenGL code reads and processes for animating the model.

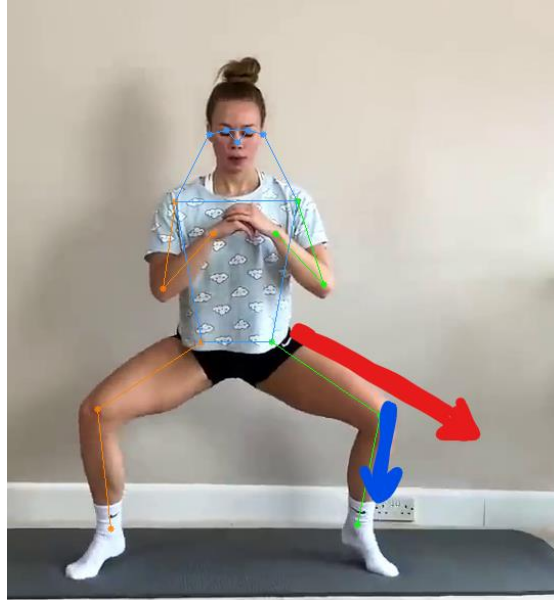


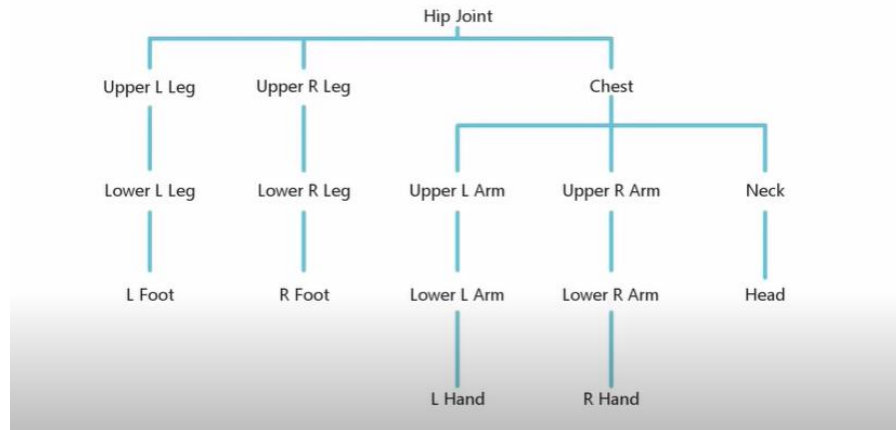
Figure 2: Obtaining a rotation matrix that rotates the red vector to blue vector in 3D space will give us the 3D orientation of lower leg with respect to the other leg.

Model Animation

We use a free humanoid model for demonstrating the animation using pose information. The model depicts a “Vampire” and has similar bone structure as compared to an adult human. In Figure 3, we illustrate the model with its bones and also present the bone hierarchy. For loading the model, we use the “Assimp” library which stands for the Asset Import Library.



(a)



(b)

Figure 3: a) A “vampire” model with its bones. b) Hierarchy of the bones of the model

For animating the model, various classes need to be set-up as illustrated in Figure 4, to keep the codes neatly organized. The model is stored in a “.dae” file which contains all its relevant information such as vertices, normals, UV, bone structure, bone-weights for each vertex etc. Each vertex is connected to one or more bones which contribute to move it with varying amounts. This connection information is stored in arrays called weights. Each vertex can have upto four bones that move it. Each of these bones can have a different amount of contribution to the overall movement of the bones. This is done by taking a weighted average of the transformations done by each bone affected by a particular vertex. We read the sequence of pose angle information (quaternions) from a text file inside the constructor of the “Joint” or “Bone” class. Then, using this information we can calculate the local transformation of each bone with respect to its parent bone at each time-step. The “Joint” (Bone) class also contains the functions of interpolating between the rotation angles for two neighboring time-steps.

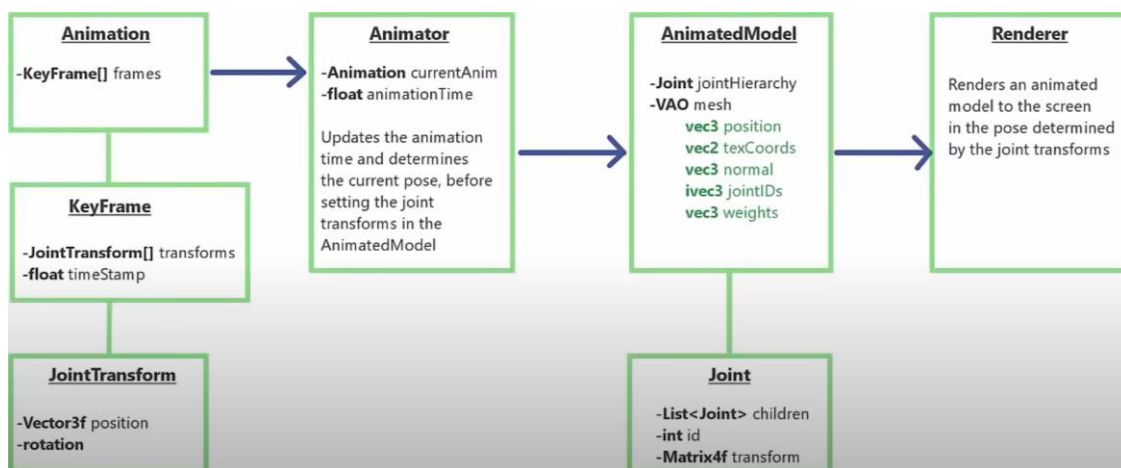


Figure 4: Classes used for Animating the Model

Results

In Figure 5 and 6, I illustrate the output of the OpenGL animation program. We can see that the model successfully mimics the human pose. The model stays in one place as I couldn't use any translation information as the Pose Detector outputs the 3D pose co-ordinates on a fixed frame with respect to the human body. Still, just using the rotation angles of each bone, the graphical model produces similar movements as seen in the input video.

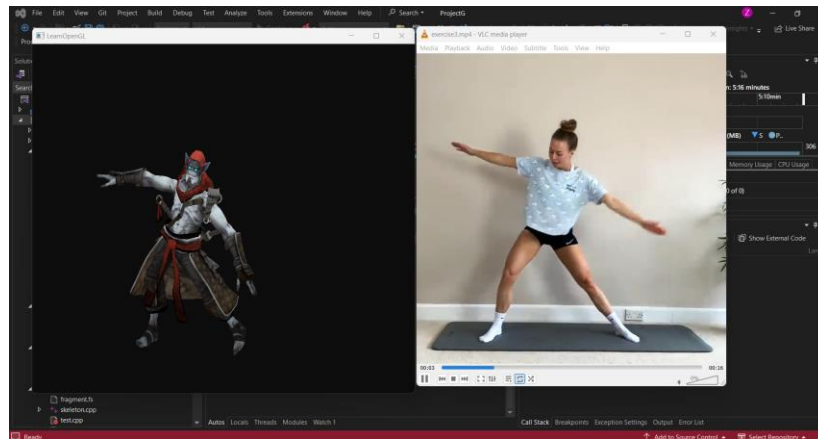


Figure 5: Output of OpenGL skeletal animation using 3D pose information

However, there are some limitations. As the body proportions of the human in the video and the vampire model are not the same, some of the poses will not be translated properly. Due to this reason, sometimes the hands of the vampire model go inside the other body parts which makes the motion unnatural. Another limitation is that the 3D poses detection pipeline outputs the co-ordinates for only 17 joints, whereas the vampire model has 65 bones. So, the expression of movement is limited using pose-detection, whereas it is possible to animate all the 65 bones if done so manually.

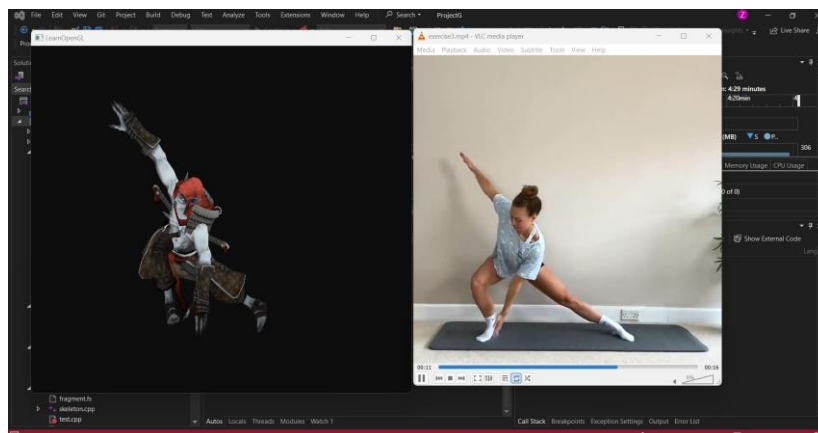


Figure 6: Output of OpenGL skeletal animation using 3D pose information

Overall, this OpenGL project illustrates the potential of computer vision AI based body-pose detectors for automating motion tracking and capture which can save a lot of time and resources of CGI and animation industry.

Online Materials Used

Documentations, tutorials, and starter codes from the following online resources were heavily utilized in this project.

<https://www.youtube.com/watch?v=f3Cr8Yx3GGA&t=605s>

<https://www.youtube.com/watch?v=GZQkwx10p-8&t=5181s>

https://mmpose.readthedocs.io/en/latest/user_guides/inference.html

<https://motionbert.github.io/>

<https://learnopengl.com/Guest-Articles/2020/Skeletal-Animation>

https://temugeb.github.io/python/motion_capture/2021/09/16/joint_rotations.html

The following youtube videos were utilized as input data-

<https://www.youtube.com/watch?v=9g4z0gFlfpA&t=480s>

<https://www.youtube.com/watch?v=WTUruNwUMFI&t=319s>

References

[1] MotionBERT: A Unified Perspective on Learning Human Motion Representations, Zhu, Wentao and Ma, Xiaoxuan and Liu, Zhaoyang and Liu, Libin and Wu, Wayne and Wang, Yizhou, Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023

[2] OpenMMLab Pose Estimation Toolbox and Benchmark, MMPose Contributors, <https://github.com/open-mmlab/mmpose>, 2023